Brickts SQL course — Week 5 — Calculate runtime and energy

sparql-playground / lessons/sql

Generated for offline / Kindle reading

Week 5 — Calculate runtime and energy

This pack concatenates the six daily lessons and the weekly challenge, then the Week primary-reading links from SOURCES.md. Work the SQL in `psql` against `brickts_course`; keep notes under ignored `student-work/sql/`.

Contents

- Day 29: Day 29 — Measure quality before calculating performance
- Day 30: Day 30 — Turn irregular samples into bounded intervals

- Day 31: Day 31 — Distinguish fan command from observed status
- Day 32: Day 32 — Calculate runtime with SQL and explicit unknown time
- Day 33: Day 33 — Integrate kW into observed kWh
- Day 34: Day 34 — Group reporting periods in each site's timezone
- Day 35 (weekly challenge): Day 35 — Week 5 independent challenge — an observed runtime and energy report

---

Day 29 — Measure quality before calculating performance

Previous: Day 28 · **Week 5 · 45–75 minutes.** Prerequisites: Days 1–28, or equivalent SQL skills.

Goal

Separate expected samples, received rows and usable measurements before reporting HVAC coverage.

Before you start

Use the disposable `brickts_course` database from the lab guide. Run `uv run python lessons/sql/lab.py check`. Read the fixture contract; keep the main fixture unchanged. Today's reporting window is `[2026-01-05 00:00Z, 2026-01-07 00:00Z)`.

Concept

The grain of `course.sample` is one point at one instant within one site. A missing row and a present row with a null value are different observations. Quality describes whether a received value may enter a calculation. This course accepts only `quality = 'good' AND value IS NOT NULL`.

At five minute cadence, each point has 576 expected instants in the window. A ratio using received rows measures something different from a ratio using those 576 expected instants. Write the denominator beside every percentage. A missing record cannot be counted by filtering existing records alone; start from the expected point/time grid or an independently calculated expectation.

Tiny example

```
SELECT count(*) AS received,
       count(value) AS nonnull,
       count(*) FILTER (WHERE quality =
         'good' AND value IS NOT NULL) AS
         usable
FROM (VALUES (12.0, 'good'), (NULL,
         'bad'), (9.0, 'bad')) AS
         v(value, quality);
```

Predict `3`, `2`, `1`. The bad numeric value explains why `count(value)` is insufficient.

SQL challenge

Report each point's site, equipment UUID, role, expected count, received count, usable count and usable percentage. Use `course.point` as the inventory so a point with zero received rows would still appear. Keep counts as integers and percentages as numeric values with named units.

Experiment

In a scratch `VALUES` relation, compare one absent sample, one bad/null sample and one bad numeric sample. Explain which totals distinguish all three. Do not insert these cases into `course.sample`.

Acceptance checks

- Inventory has 42 points and 24,192 expected rows.
- Fixture has 24,190 received rows and 24,189 usable measurements.
- North Supply A has 574 received pressure samples and 575 usable power samples. Its power point still has 576 received rows.

Stretch and reflection

Would a sensor reporting twice as frequently deserve twice the statistical weight? Explain why count coverage becomes inadequate when cadence changes.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day29/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 28 ·

Day 30 — Turn irregular samples into bounded intervals

Previous: Day 29 · **Week 5 · 45–75 minutes.** Prerequisites: Days 1–29, or equivalent SQL skills.

Goal

Calculate observed seconds without carrying one measurement through an arbitrarily long outage.

Before you start

Keep Day 29's quality policy and half-open window. Save a separate interval query; later runtime, energy and fault reports will reuse its declared convention.

Concept

A sample is an instant. Duration is an interpretation. For this course, a sample's value represents the interval beginning at its timestamp and ending at the earliest of the

next sample, 300 seconds later, or the reporting end. Clip its beginning to the reporting start. Only positive intervals contribute. Partition `lead(ts)` by site and point and order by time; crossing a point boundary creates meaningless durations.

Choose the immediate next received timestamp before excluding bad rows. A bad observation ends the earlier observation's authority and starts unknown time. When a window begins between observations, inspect a predecessor up to 300 seconds before the start; filtering to the window before computing intervals loses that possible coverage. Absence beyond the cap remains unknown.

Tiny example

```
SELECT ts, lead(ts) OVER (ORDER BY ts) AS
         next_ts
FROM (VALUES ('2026-01-05
         08:20Z'::timestamptz),
             ('2026-01-05
         08:35Z'::timestamptz)) AS v(ts);
```

The first successor is fifteen minutes away. Under the five minute cap, its covered interval ends at 08:25, leaving ten minutes unknown.

SQL challenge

Build intervals for north Supply A's `duct_static` point using its discovered UUID. Return interval start, end, duration seconds and eligibility. Calculate valid coverage and unknown seconds for the 48 hour window. Keep the last sample's bounded tail; a null `lead` does not automatically mean zero duration.

Experiment

Use three scratch timestamps at 00:00, 00:02 and 00:12. Predict bounded coverage for `[00:01,00:13)` before running. Compare a 300 second cap with unrestricted forward fill and explain the difference.

Acceptance checks

- Full fixture pressure coverage is 47 hours 50 minutes, with 600 unknown seconds.
- No interval lasts more than 300 seconds; adjacent intervals do not overlap.
- Moving the window boundary inside a valid interval clips that interval accurately.

Stretch and reflection

How would a counter, an instantaneous flow measurement and a discrete status differ in their appropriate interpolation policy? Keep today's policy explicit rather than assuming it is universal.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day30/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 29 ·

Day 31 — Distinguish fan command from observed status

Previous: Day 30 · **Week 5 · 45–75 minutes.** Prerequisites: Days 1–30, or equivalent SQL skills.

Goal

Define an auditable fan operating state, including disagreement and unavailable evidence.

Before you start

Inspect `course.point` roles and units. The fixture contains `fan_status` and `fan_speed_pct`; it has no independent binary fan command point. A speed percentage is a requested operating signal only when the adapter explicitly assigns that meaning.

Concept

A command describes intent; status describes reported operation. Neither proves airflow. For this teaching policy, eligible binary status takes precedence: `1` means on, `0` means off. If status is unavailable, retain unknown for observed runtime. A separate command-based estimate can be reported with a different name. Do not silently replace missing status with speed or turn null into off.

Treat a disagreement as evidence worth preserving. An on command with off status could indicate a delayed start, a safety trip, a mapping error or stale data. SQL alone cannot choose among those explanations. State values outside the declared binary domain are invalid, even when quality says good.

Tiny example

```
SELECT status, command,
       CASE WHEN status = 1 THEN 'on'
            WHEN status = 0 THEN 'off'
            ELSE 'unknown' END AS
         observed_state
FROM (VALUES (0, 1), (1, 0), (NULL, 1)) AS
         v(status, command);
```

This three-row example is deliberately independent of the fixture. Its states are off, on and unknown; the first two rows also

disagree with command.

SQL challenge

Create a state projection over `course.ahu_5m`, retaining site, equipment UUID and timestamp. Name your status validity rule. Add a separate speed-request indicator using a threshold you declare, without presenting it as measured runtime. Count observed on, off, unknown and command/status disagreements per equipment.

Experiment

Construct scratch rows for status `NULL`, `2`, `0` and `1`, paired with speed values below, at and above your threshold. Predict the classification at the threshold. Compare policies using `>` and `>=`.

Acceptance checks

- Binary status produces no unknown fixture states.
- Both sites' equipment called Supply A remain distinct.
- Scratch missing or invalid status stays unknown; disagreement is visible.
- Your report names which signal supports observed operation and which supports

an estimate.

Stretch and reflection

What evidence would you require before using current, differential pressure or speed feedback as a runtime proxy? Record one failure mode for each possible substitute.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day31/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 30 ·

Day 32 — Calculate runtime with SQL and explicit unknown time

Previous: Day 31 · **Week 5 · 45–75 minutes.** Prerequisites: Days 1–31, or equivalent SQL skills.

Goal

Sum observed fan-on seconds while keeping off time and unavailable time visible.

Before you start

Reuse the bounded interval convention from Day 30 and the state policy from Day 31. Use UTC for today's full 48 hour report, with duration expressed in seconds and hours.

Concept

Runtime is the sum of eligible interval seconds whose observed state is on. It is not generally the count of on rows divided by twelve. That shortcut works only when every

selected row represents exactly five minutes, with correct edges and no duplicates or quality gaps.

The reporting grain is one equipment and window. On, off and unknown seconds should reconcile to the window's duration. Unknown means the available evidence does not support either state. A schedule describes intended hours; it is a separate quantity even when it matches the fixture's observed status.

Tiny example

```
SELECT sum(seconds) FILTER (WHERE state =
         'on') / 3600.0 AS on_hours,
       sum(seconds) FILTER (WHERE state =
         'unknown') AS unknown_seconds
FROM (VALUES ('on', 120), ('on', 300),
             ('off', 300), ('unknown',
        180)) AS v(state, seconds);
```

The two on intervals total seven minutes. Counting each as five minutes would overstate runtime.

SQL challenge

Produce one runtime row per equipment using eligible `fan_status` intervals. Include site and equipment identity, requested window boundaries, on/off/unknown seconds, hours

and valid coverage percentage. Compare observed hours with the contract's scheduled hours in a separately labelled column.

Experiment

Repeat for `[2026-01-05 05:58Z, 2026-01-05 06:07Z)` on north Supply A. Account for the predecessor and the window end. Predict two off minutes and seven on minutes. Explain why merely selecting sample timestamps within the window loses information.

Acceptance checks

- Over the 48 hour fixture: north Supply A has 24 on hours, north Supply B 12, and south Supply A 12.
- Each equipment has zero unknown status seconds and 48 total classified hours.
- The partial-window experiment reconciles to nine minutes.
- Empty on time is represented as zero; unavailable data remains separately identifiable.

Stretch and reflection

Would resetting a controller counter or losing a status sensor change the schedule? Explain how your output prevents those separate facts

from being conflated.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day32/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 31 ·

Day 33 — Integrate kW into observed kWh

Previous: Day 32 · **Week 5 · 45–75 minutes.** Prerequisites: Days 1–32, or equivalent SQL skills.

Goal

Calculate electrical energy from bounded power intervals and explain the effect of one bad reading.

Before you start

Use the same UTC boundaries and 300 second maximum hold as runtime. Read the power unit from `course.point`; it is kW. Do not assume a role name alone guarantees units in another dataset.

Concept

Power is a rate. Energy for a constant interval is `kW × seconds / 3600`, producing kWh. Summing kW values yields neither a mean power nor an energy total. This lesson uses a left-held rectangle, with each eligible power

sample held through its bounded interval. A trapezoid needs two eligible endpoints and a different gap policy; it must not appear silently inside the same report.

Power coverage differs from status coverage. A fan may be known to run while its power reading is unavailable. Preserve that missing energy interval. An estimate derived from the schedule or neighboring readings belongs in a separately labelled field with its own assumptions.

Tiny example

```
SELECT sum(kw * seconds / 3600.0) AS
         observed_kwh
FROM (VALUES (12.0, 300), (6.0, 120)) AS
         v(kw, seconds);
```

The result is 1.2 kWh. Here, the average of 12 and 6 kW would not correctly weight the unequal interval lengths.

SQL challenge

Integrate eligible `kw` samples per equipment. Return observed kWh, valid power seconds, unknown power seconds and a coverage percentage. Calculate scheduled energy

separately from the contract's stated power and schedule, and name that result as an estimate.

Experiment

Inspect the north Supply A reading at Jan 5 15:10Z. Compare excluding that interval, replacing null with zero and interpolating it. All three can produce a number; explain which claim each number supports. Keep experimental replacements in scratch data.

Acceptance checks

- North Supply A's observed energy is 287 kWh, with 300 unknown power seconds. Scheduled energy is 288 kWh.
- North Supply B has 72 kWh; south Supply A has 108 kWh.
- A missing interval is not reported as known zero consumption.
- Output retains units and the integration policy.

Stretch and reflection

Explain why 287 kWh is an observed subtotal under this policy. Which additional evidence would justify estimating the missing 1 kWh?

Save SQL, actual output and a short interpretation in your own `student-work/sql/day33/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 32 ·

Day 34 — Group reporting periods in each site's timezone

Previous: Day 33 · **Week 5 · 45–75 minutes.** Prerequisites: Days 1–33, or equivalent SQL skills.

Goal

Build local calendar reporting periods while preserving UTC instants and elapsed durations.

Before you start

Inspect `course.site.tz`; use the stored IANA timezone names. Set a display timezone only if helpful, and record it. The fixture spans 48 UTC hours and may cover partial local dates or weeks.

Concept

A `timestamptz` identifies an instant. `ts AT TIME ZONE site.tz` gives its local wall-clock representation. Local week labels describe a

calendar, while energy and runtime remain elapsed seconds. PostgreSQL's `date_trunc('week', ...)` uses a Monday start. State that convention explicitly.

Do not assign an entire interval to the week containing its starting sample if the interval crosses a week boundary. Split intervals at local boundaries converted back to UTC, then aggregate the pieces. Daylight saving transitions can make a local day shorter or longer than 24 hours. A partial week in this two-day fixture does not become a complete week because its label is Monday.

Tiny example

```
SELECT ts,
       ts AT TIME ZONE 'America/Chicago'
         AS local_time,
       date_trunc('week', ts AT TIME ZONE
         'America/Chicago') AS local_week
FROM (VALUES ('2026-01-05
         00:00Z'::timestamptz)) AS v(ts);
```

That instant is Sunday evening locally, so its local week starts on the preceding Monday. Changing the session display timezone does not change the underlying instant.

SQL challenge

Extend your runtime and energy reports with site-local week labels, UTC period boundaries, requested-window intersection and a partial-period flag. Preserve each equipment's identity. Compare local-week grouping with UTC-week grouping and explain any extra group.

Experiment

Use scratch instants around a spring daylight saving transition in a timezone that observes it. Calculate elapsed seconds between consecutive local midnights converted to UTC. Predict the result before running; do not alter fixture timestamps.

Acceptance checks

- Your one-instant demonstration places Jan 5 00:00Z in the previous Chicago calendar week.
- Every covered interval belongs to exactly one period after splitting.
- Summing local-period observed runtime and kWh reproduces Days 32–33 totals.
- No partial period claims 168 hours of fixture coverage.

Stretch and reflection

How should a dashboard label a weekly total when the requested data covers only Sunday evening? Propose a label containing both the calendar week and actual coverage.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day34/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 33 ·

Day 35 — Week 5 independent challenge — an observed runtime and energy report

Previous: Day 34 · **Week 5 review · 2–4 hours, split across sessions as needed.** Prerequisites: Days 29–34, or equivalent skills.

Project brief

Deliver a reproducible report for all three synthetic AHUs over the contract's complete 48 hour window. An operator should be able to distinguish observed fan runtime, observed energy, schedule estimates and unknown coverage. Include a second report grouped by site-local week. Work only in the disposable course database; preserve the starter fixtures.

Acceptance criteria

- One summary row per site/equipment UUID includes window, units, duration convention and signal precedence.

- Observed fan-on hours reconcile to 24, 12 and 12 for north Supply A, north Supply B and south Supply A respectively.
- Observed energy across the three AHUs totals 467 kWh; scheduled energy totals 468 kWh. Explain the difference using evidence.
- Power unknown time is visible, with no invented energy value. On/off/unknown status seconds reconcile to each equipment window.
- Local weekly pieces reconcile to the full-window totals and expose partial periods.
- A separate partial-window case proves clipping and predecessor handling.

Deliverables

Submit SQL, compact result tables and a one-page interpretation. Include one failed or misleading calculation you corrected, its consequence and a reproducible counterexample. Keep your independently written implementation separate from earlier tiny examples.

Self-review

Could somebody mistake an estimate for a measurement? Which rule limits forward holding? How does your report behave if

status is known while power is missing? Explain each answer using a specific output field.

Label results **observed**, **fixture-only**, or **not run**. This page intentionally provides no worked solution or implementation sequence. Attempt the challenge independently before consulting the week's references.

Previous: Day 34 ·

Primary reading — Week 5

Use the lesson first, then these references for precise behavior. Full list: SOURCES.md.

Week 5

- Date/time functions
- Aggregate functions
- Window function frames
- Open-FDD source navigation